

B-Tree vs B+ Tree Architecture

Multilevel Indexing, Structural Comparison & Search Mechanism Fundamentals

1. WHY MULTILEVEL INDEXING?

Database systems organize index records in balanced search trees to accelerate search operations in secondary memory.

CONCEPT Index Record Components

-  **Key:** The search attribute value (e.g., Roll No, ID).
-  **Data Pointer:** Address pointing to actual record in memory.
-  **Block Pointer:** Address pointing to child node.

ANALOGY Book Index Model

Index Page → Key (Topic Name) + Pointer (Page Number)

↓

Actual Data → Full Topic Content on Page

2. B-TREE STRUCTURE

Core Invariant of B-Tree

In a B-Tree, every node (Root, Internal, and Leaf) follows the **exact same structure** containing Keys, Data Pointers, and Child Block Pointers.

B-Tree Node Uniform Layout

[Ptr₁ | Key₁ | DataPtr₁ | Ptr₂ | Key₂ | DataPtr₂ | Ptr₃]

*At Leaf level, child block pointers (Ptr_i) are set to **NULL**, but structural fields remain present.*

3. B+ TREE STRUCTURE

B+ Tree separates routing logic from record access by introducing two distinct node types:

1. Internal / Non-Leaf Node

Contains **ONLY Keys + Child Block Pointers**.

[BlockPtr | Key | BlockPtr]

✗ No Data Pointers. Maximizes node capacity (fan-out).

2. Leaf Node

Contains **Keys + Data Pointers + Next Leaf Link**.

[Key | DataPtr] → [Key | DataPtr] →
Next

✓ All actual data pointer references live here.

4. STRUCTURAL FAN-OUT & SEARCH TRAVERSAL

Why B+ Tree is Broader (Less Depth)

Since internal nodes omit Data Pointers, more space remains for Block Pointers. This increases the **Fan-out Ratio**, allowing more keys per node and reducing tree height.

Search Paths

B-Tree: Search can terminate early at root or internal levels.

B+ Tree: All searches proceed uniformly down to the leaf level.

5. KEY DUPLICATION & LINKED LEAVES IN B+ TREE

MECHANISM Navigation Key Copying

In B+ Trees, internal node keys act strictly as **navigation routers**. A key in an internal node is duplicated at the leaf level alongside its corresponding Data Pointer.

FEATURE Leaf Node Linked List

All leaf nodes are connected sequentially using pointers forming a singly linked list:

Leaf₁ → Leaf₂ → Leaf₃

Range Query Efficiency

Example: Find roll numbers 100 to 150.

In B+ Tree, search navigates to key 100 at the leaf level, then sequentially traverses linked leaves directly (100 → 101 → 102 ... → 150) without re-traversing upper tree levels.

6. COMPLETE COMPARISON MATRIX

FEATURE	B-TREE	B+ TREE
Data Pointer Location	Root, Internal, and Leaf nodes	Leaf nodes ONLY
Internal Node Contents	Key + Data Pointer + Block Pointer	Key + Block Pointer
Node Structure	Identical at all levels	Differs (Internal vs Leaf)
Key Duplication	No redundant keys	Keys duplicated in Leaf nodes
Leaf Linkage	Leaf nodes are NOT linked	Leaf nodes form a Linked List
Range Queries	Inconvenient (Tree Traversal needed)	Extremely fast & efficient
Capacity / Fan-out	Lower (Data Pointers consume space)	Higher (More children per node)
Tree Height / Shape	Relatively higher depth	Relatively broader / lower depth

7. EXAM TRAPS & HIGH-YIELD PITFALLS

⚠ Trap 1: Internal Data Pointers

Do B+ Tree internal nodes hold data pointers? **NO**. Data pointers exist solely in leaf nodes.

⚠ Trap 2: Uniform Node Layout

Does B-Tree use different node formats? **NO**. B-Tree nodes are uniform; B+ Tree node structures differ.

8. SUMMARY REVISION CHEATSHEET

🌳 B-TREE: DATA EVERYWHERE

- Root → Keys + Data
- Internal → Keys + Data
- Leaf → Keys + Data
- Leaves NOT connected

🌳 B+ TREE: DATA AT LEAVES

- Internal → Navigation Keys Only
- Leaf → Keys + Data Pointers
- Leaves connected via Linked List
- Optimized for Range Scanning